# Taskaya — Phase 2 Remaining Work & Bonus Split

Based on `Mobile_Application_Development_Project_Form.docx` and the current codebase.

**Goal:** complete all remaining Phase 2 requirements, preserve the finished Phase 1 experience, and turn the existing dark mode + animations into the submitted bonus features.

---

## 1. Current Status

### Already complete

- Phase 1 task CRUD, completion toggle, navigation, and polished UI
- Task priority and due date/time
- Form validation for authentication and task title
- Dark mode with persisted preference
- Screen/list/checkbox animations with reduced-motion support
- In-app reminder inbox (not an operating-system notification)

### Still required

- Firebase Authentication (email/password)
- Cloud Firestore task storage
- Per-user task isolation
- Search and completed/pending filtering

- Firebase/network error handling
- Phase 2 report and release APK

### Bonus status

- **Complete:** Dark Mode, Animations
- **Optional extra bonus work:** Task Categories and true Local Notifications
- **Not implemented:** Voice Input

---

## 2. Ownership

| Area | Owner | Main outcome |
|---|---|---|
| Firebase setup, Authentication, local notifications | Mostafa | Real user accounts, secure sign-in/session handling, scheduled device reminders |
| Firestore tasks, search/ filter, categories | Ahmed | Cloud-synced, user-scoped task management and discovery controls |
| Security rules, integration QA, APK and report | Shared | A safe, demonstrable, submittable Phase 2 release |

---

## 3. Ahmed — Task Data & Discovery

### A. Replace the local task repository with Cloud Firestore

**Files affected:** `lib/data/repositories/task_repository.dart`, `lib/providers/task_provider.dart`,

`lib/data/models/task.dart`

- ☐ Create a Firestore-backed task repository.
- ☐ Store each task under its authenticated owner, for example: `users/{uid}/tasks/{taskId}`.
- ☐ Convert task reads to a live stream so another device/session updates the list automatically.
- ☐ Make create, edit, delete, restore, and complete/incomplete operations write to Firestore.
- ☐ Add loading and operation-failure states to `TaskProvider`.
- ☐ Preserve priority, due date/time, description, completion state, created date, and completed date in Firestore.
- ☐ Remove the seeded in-memory task list from the production path; keep only an optional debug/demo seed if needed.

**Acceptance check:** create a task, restart the app, sign in again, and verify that the task still exists and only appears for its owner.

## B. Search tasks

**Files affected:** `lib/screens/home/home_screen.dart`, `lib/providers/task_provider.dart`

- ☐ Add a search field to Home.
- ☐ Search task title and description, case-insensitively.
- ☐ Search across both pending and completed tasks.
- ☐ Add a clear-search action and a useful no-results state.
- ☐ Ensure search works with Firestore-loaded tasks and does not break task animations.

**Acceptance check:** searching a unique word shows only matching tasks; clearing the query restores the normal list.

## C. Filter completed and pending tasks

- ☐ Add a visible filter control: **All** / **Pending** / **Completed**.
- ☐ Keep the existing Pending and Completed sections when “All” is selected.

- ☐ Show a clear empty state when the chosen filter has no results.
- ☐ Make search and filtering work together.

**Acceptance check:** every filter returns the correct tasks, including after a task is marked complete or incomplete.

### D. Optional bonus: Task Categories

- ☐ Add a `TaskCategory` model/field (for example: Personal, Study, Work, Other).
- ☐ Add category selection to Add/Edit Task.
- ☐ Persist categories in Firestore.
- ☐ Show the category on task cards or support category filtering.

**Acceptance check:** category survives edit, app restart, and sign-in on another device.

---

## 4. Mostafa — Firebase Auth, Reliability & Notifications

### A. Configure Firebase

**Files affected:** `pubspec.yaml`, Android/iOS Firebase configuration, `lib/main.dart`

- ☐ Create/connect the Firebase project for Android and iOS.
- ☐ Add and configure `firebase_core`, `firebase_auth`, and `cloud_firestore`.
- ☐ Add generated Firebase configuration files without exposing secrets incorrectly.
- ☐ Initialize Firebase before the app starts.
- ☐ Verify Android build configuration is compatible with Firebase.

**Acceptance check:** the app starts successfully on Android with Firebase initialized.

## B. Replace mock authentication with Firebase Email/ Password Auth

**Files affected:** `lib/data/repositories/auth_repository.dart`, `lib/data/repositories/mock_auth_repository.dart`, `lib/providers/auth_provider.dart`, login/signup screens

- ☐ Create `FirebaseAuthRepository` implementing the existing `AuthRepository` interface.
- ☐ Register with email/password and store a display name where appropriate.
- ☐ Log in with Firebase email/password.
- ☐ Restore the Firebase session on splash screen.
- ☐ Log out from Firebase.
- ☐ Remove the visible demo-account hint from the production login screen.
- ☐ Connect the repository used in `main.dart` to Firebase rather than `MockAuthRepository`.

**Acceptance check:** a newly created account can log out, restart the app, log back in, and reaches only its own task list.

## C. Proper Firebase and network error handling

- ☐ Map Firebase Auth errors to friendly messages (invalid credentials, duplicate email, weak password, offline/network failure, and unexpected failure).
- ☐ Add retry/error states for Firestore loading and writes in collaboration with Ahmed.
- ☐ Prevent duplicate submissions while login, signup, save, or delete is in progress.
- ☐ Keep validation messages near the relevant input and preserve the existing loading indicators.

**Acceptance check:** turning off the network produces a readable recovery message instead of a crash or endless loader.

### D. Optional bonus: True Local Notifications / Task Reminders

**Files affected:** `pubspec.yaml`, Android/iOS permissions, notification service, Add/Edit Task

- ☐ Add a local-notification package and timezone support.
- ☐ Request notification permission where required.
- ☐ Schedule a device notification when a task has a future due date/time.
- ☐ Cancel and reschedule the reminder after task edits, completion, or deletion.
- ☐ Open the relevant task when the user taps its notification.
- ☐ Keep the existing in-app bell as a complementary reminder inbox, not as the only notification feature.

**Acceptance check:** close/background the app and confirm a scheduled reminder appears at the selected time.

---

## 5. Shared Finalization

### Firestore security and user isolation

- ☐ Write Firestore security rules so users can read and write only `users/{theirUid}/tasks/*`.
- ☐ Test with two Firebase accounts: neither account can see or modify the other account's tasks.
- ☐ Confirm unauthenticated users cannot access task documents.

### End-to-end QA

- ☐ Run `flutter analyze` with no issues.
- ☐ Test login, signup, logout, session restore, CRUD, completion, search, filters, priority, due date/time, and errors.

- ☐ Test on a physical Android device in light and dark modes.
- ☐ Confirm data persists after restart and across login sessions.
- ☐ Re-test notification scheduling after add, edit, delete, and complete actions if the optional local-notification feature is implemented.

### Submission deliverables

- ☐ Build the final release APK: `flutter build apk --release`.
- ☐ Add the APK to the submission package.
- ☐ Create the Phase 2 PDF report: overview, Firebase integration, screenshots, team contributions, and challenges.
- ☐ Include screenshots of authentication, task list, search/filter, add/edit, Firestore-backed persistence, dark mode, and any completed bonus feature.
- ☐ Zip the final Flutter project folder and verify it builds from a clean checkout.

---

## 6. Suggested Order of Work

1. **Mostafa:** Firebase project/configuration and Firebase Auth.
2. **Ahmed:** Firestore task repository and task-provider migration once the authenticated user ID is available.
3. **Ahmed:** Search and filter UI.
4. **Mostafa:** Firebase/network error handling and optional local notifications.
5. **Shared:** Firestore security rules, two-account isolation test, full QA, APK, report, and submission zip.

## 7. Definition of Done

The project is ready to submit when a real account can sign up, log in,

create/search/filter/update/delete tasks that persist in Firestore, and cannot access another user's tasks; the app handles invalid/offline states cleanly; the final APK and Phase 2 report are included. Dark mode and animations are already ready to claim for the bonus.